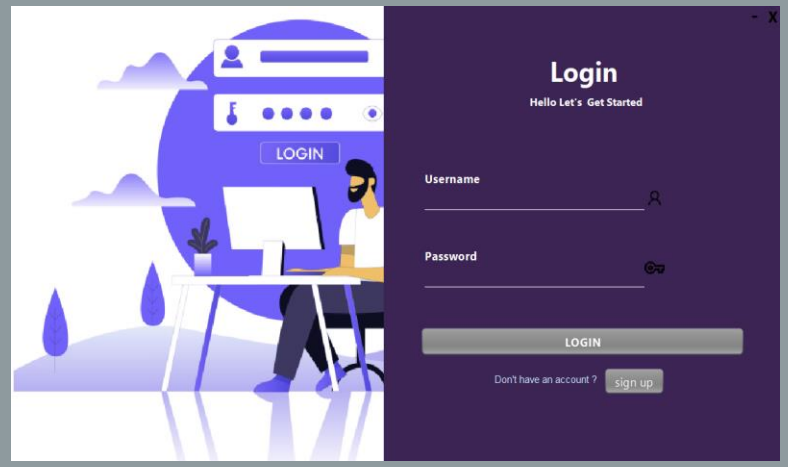
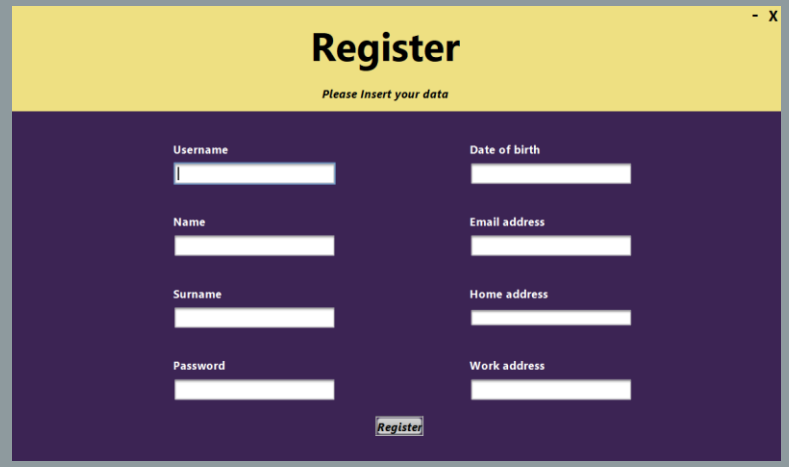
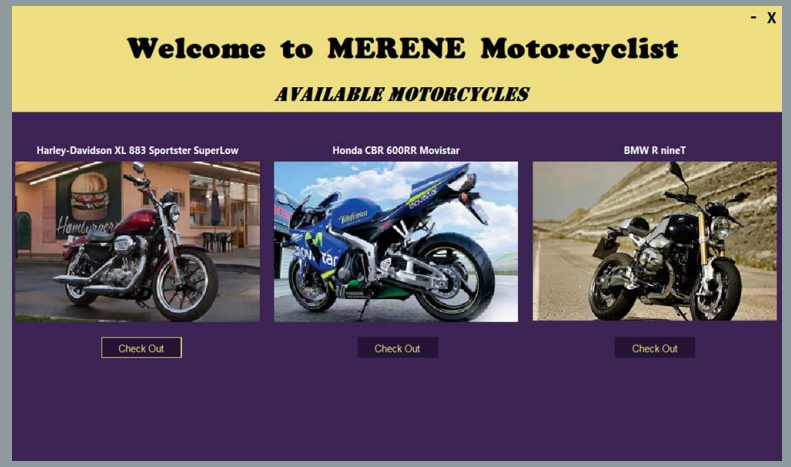
# REPORT OF E-MERENE

Name : Mehmet Mert Yigit

Number : 220315103



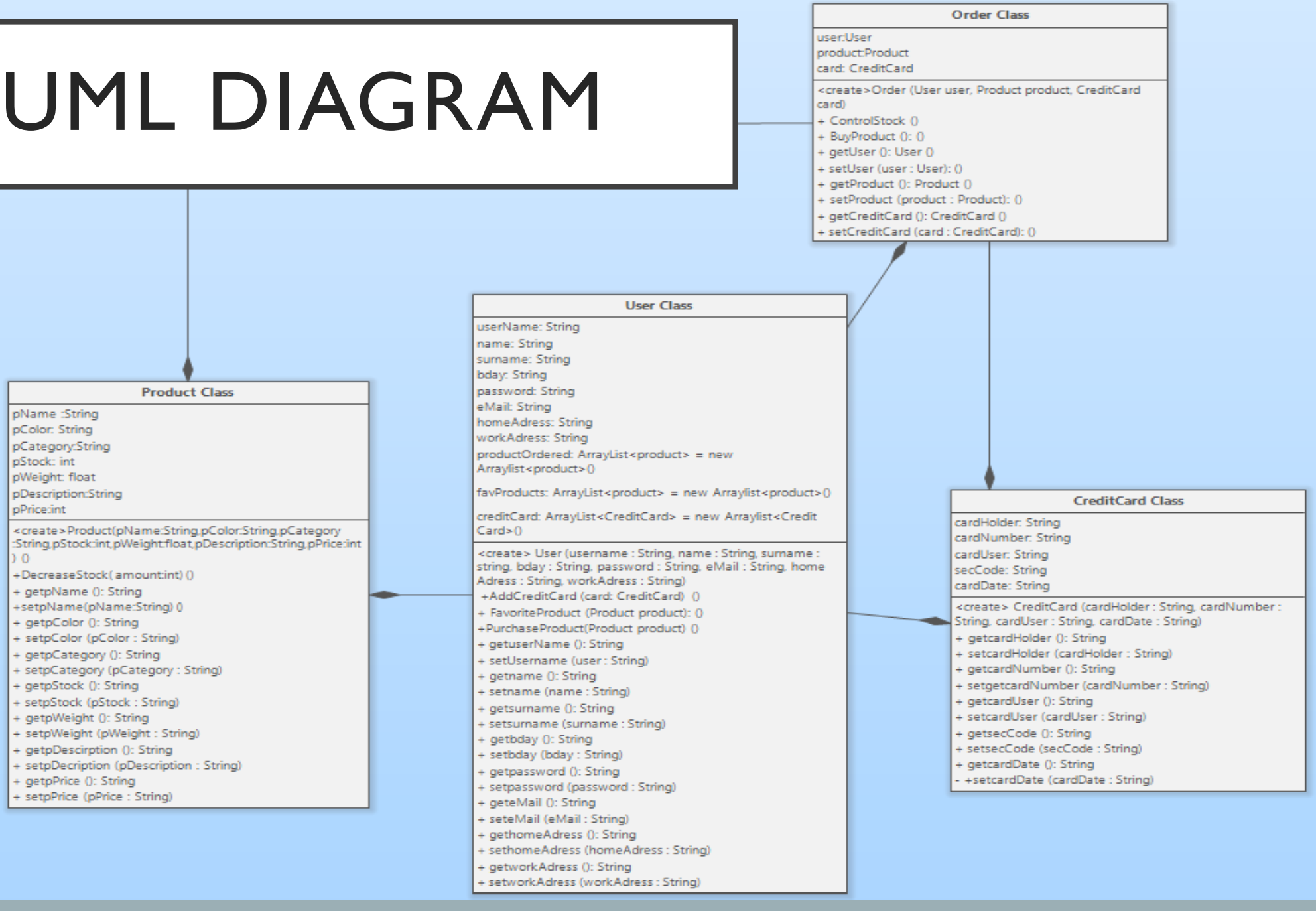




# INTRODUCTION

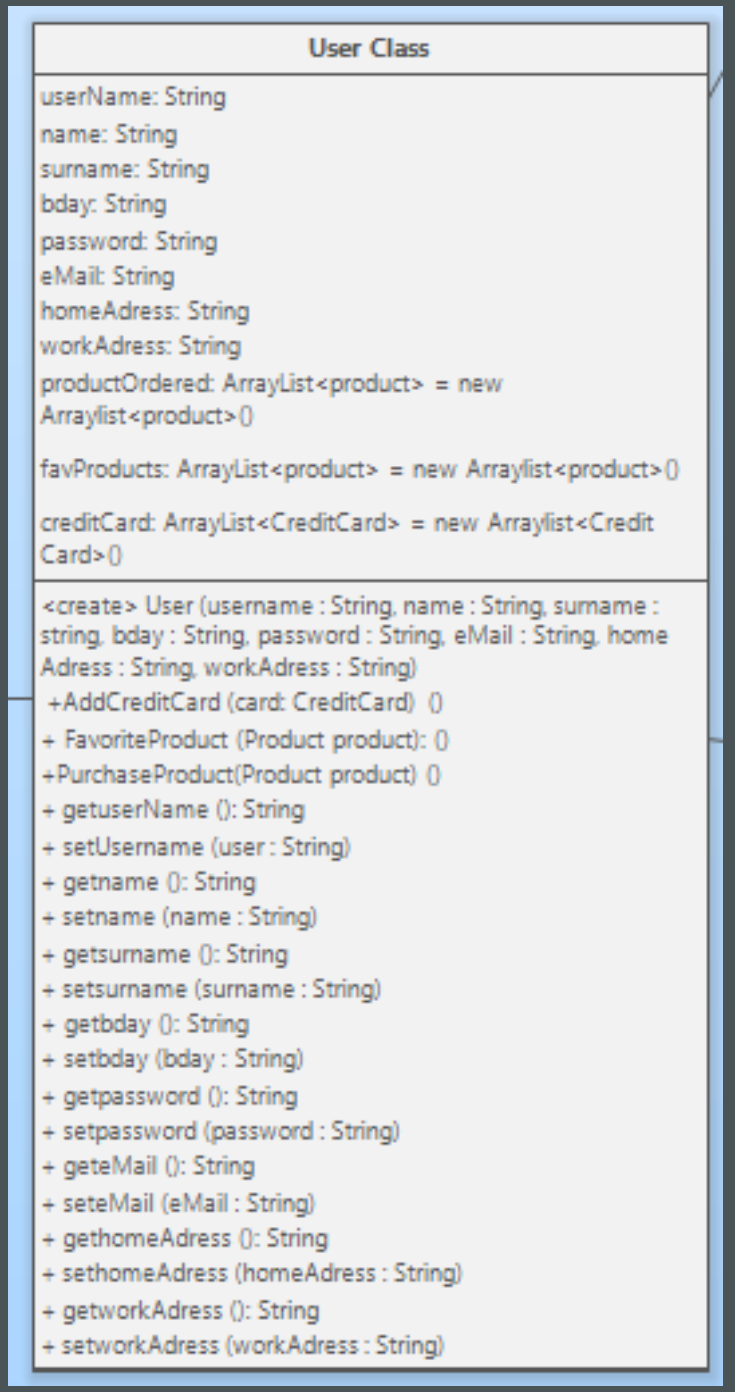
- This e-commerce application, called MERENE, specializes in selling luxury motorcycles. At present, there are three distinct motorcycles available for purchase. The application is written in Java using the NetBeans IDE. The codebase comprises User, CreditCard, Product, and Order classes, as well as several GUI classes.

# UML DIAGRAM


Order Class
user:User
product:Product
card: CreditCard
<create>Order (User user, Product product, CreditCard card)
+ ControlStock ()
+ BuyProduct (): ()
+ getUser (): User ()
+ setUser (user : User): ()
+ getProduct (): Product ()
+ setProduct (product : Product): ()
+ getCreditCard (): CreditCard ()
+ setCreditCard (card : CreditCard): ()
User Class
userName: String
name: String
surname: String
bday: String
password: String
eMail: String
homeAdress: String
workAdress: String
productOrdered: ArrayList<product> = new Arraylist<product>()
favProducts: ArrayList<product> = new Arraylist<product>()
creditCard: ArrayList<CreditCard> = new Arraylist<CreditCard>()
<create> User (username : String, name : String, surname : string, bday : String, password : String, eMail : String, home Adress : String, workAdress : String)
+AddCreditCard (card: CreditCard) ()
+ FavoriteProduct (Product product): ()
+PurchaseProduct(Product product) ()
+ getuserName (): String
+ setUsername (user : String)
+ getname (): String
+ setname (name : String)
+ getsurname (): String
+ setsurname (surname : String)
+ getbday (): String
+ setbday (bday : String)
+ getpassword (): String
+ setpassword (password : String)
+ geteMail (): String
+ seteMail (eMail : String)
+ gethomeAdress (): String
+ sethomeAdress (homeAdress : String)
+ getworkAdress (): String
+ setworkAdress (workAdress : String)
Product Class
pName :String
pColor: String
pCategory:String
pStock: int
pWeight: float
pDescription:String
pPrice:int
<create>Product(pName:String,pColor:String,pCategory :String,pStock:int,pWeight:float,pDescription:String,pPrice:int ) ()
+DecreaseStock( amount:int) ()
+ getpName (): String
+setpName(pName:String) ()
+ getpColor (): String
+ setpColor (pColor : String)
+ getpCategory (): String
+ setpCategory (pCategory : String)
+ getpStock (): String
+ setpStock (pStock : String)
+ getpWeight (): String
+ setpWeight (pWeight : String)
+ getpDescirption (): String
+ setpDecription (pDescription : String)
+ getpPrice (): String
+ setpPrice (pPrice : String)
CreditCard Class
cardHolder: String
cardNumber: String
cardUser: String
secCode: String
cardDate: String
<create> CreditCard (cardHolder : String, cardNumber : String, cardUser : String, cardDate : String)
+ getcardHolder (): String
+ setcardHolder (cardHolder : String)
+ getcardNumber (): String
+ setgetcardNumber (cardNumber : String)
+ getcardUser (): String
+ setcardUser (cardUser : String)
+ getsecCode (): String
+ setsecCode (secCode : String)
+ getcardDate (): String
- +setcardDate (cardDate : String)

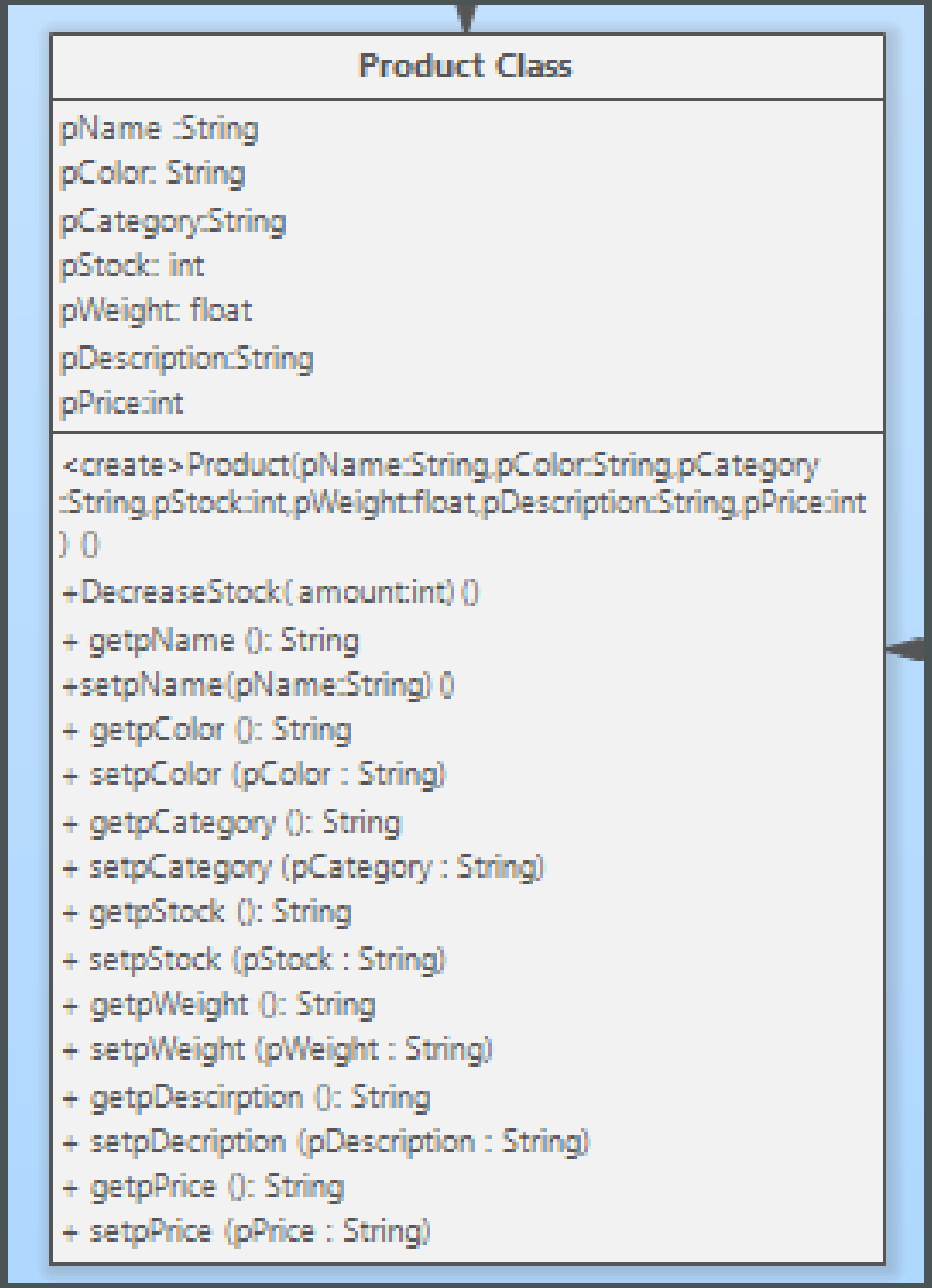
# UML DIAGRAM USER CLASS PART

User class consists of userName, name, surname,bDay, password, eMail, homeAdress and workAddress, user's credit cards, favorite products and ordered products. All these components have their own getters and setters in the class. For example, getUserName and setUserName methods are used to get and set the username, while getBday and setBday methods are used to get and set the date of birth. Additionally, this class has several methods like AddCreditCard, FavoriteProduct, UnfavoriteProduct, and PurchaseProduct.



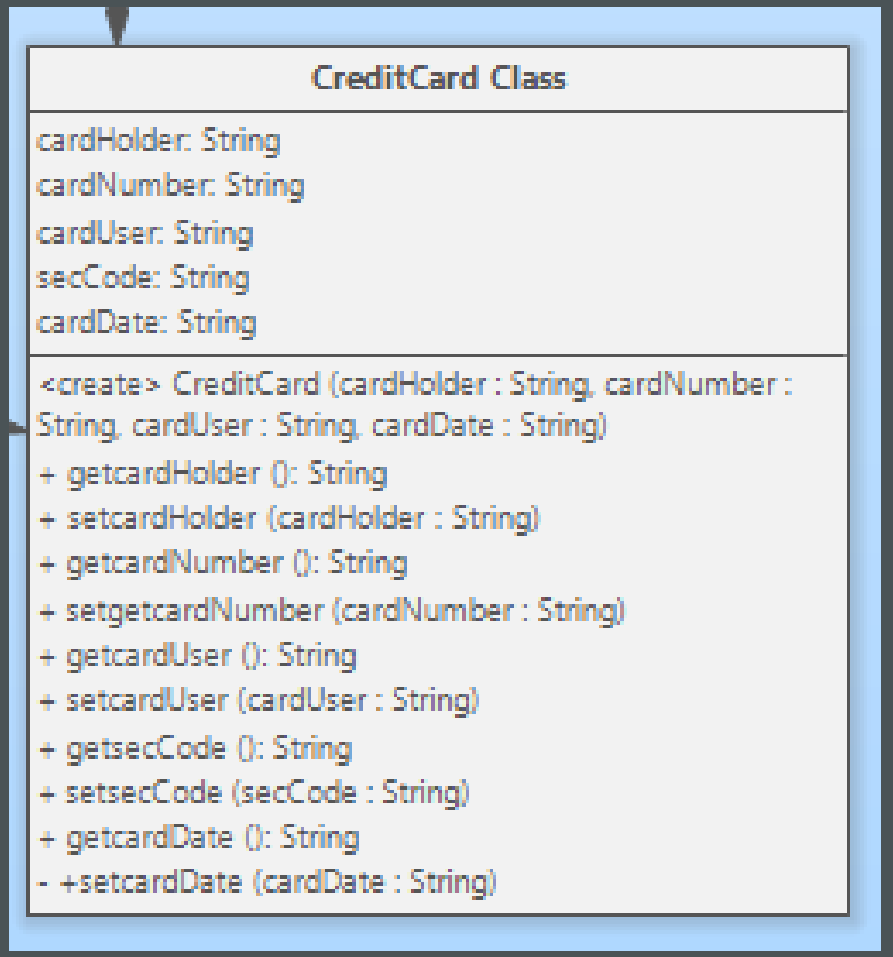
# UML DIAGRAM PRODUCT CLASS PART

Product class consists of product name, product color, product category, product stock, product weight, product description, and product price. All these components have their own getters and setters in the class. For example, getName and setName methods are used to get and set the product name, while getPrice and setPrice methods are used to get and set the product price. Additionally, this class has methods like DecreaseStock, which decreases the product stock by a specified amount, getColor and setColor to get and set the product color, getCategory and setCategory to get and set the product category, getStock and setStock to get and set the product stock, getWeight and setWeight to get and set the product weight, getDescription and setDescription to get and set the product description, and getPrice and setPrice to get and set the product price. These methods ensure that all product-related data can be accessed and modified as needed, providing a robust framework for managing product information in an application.



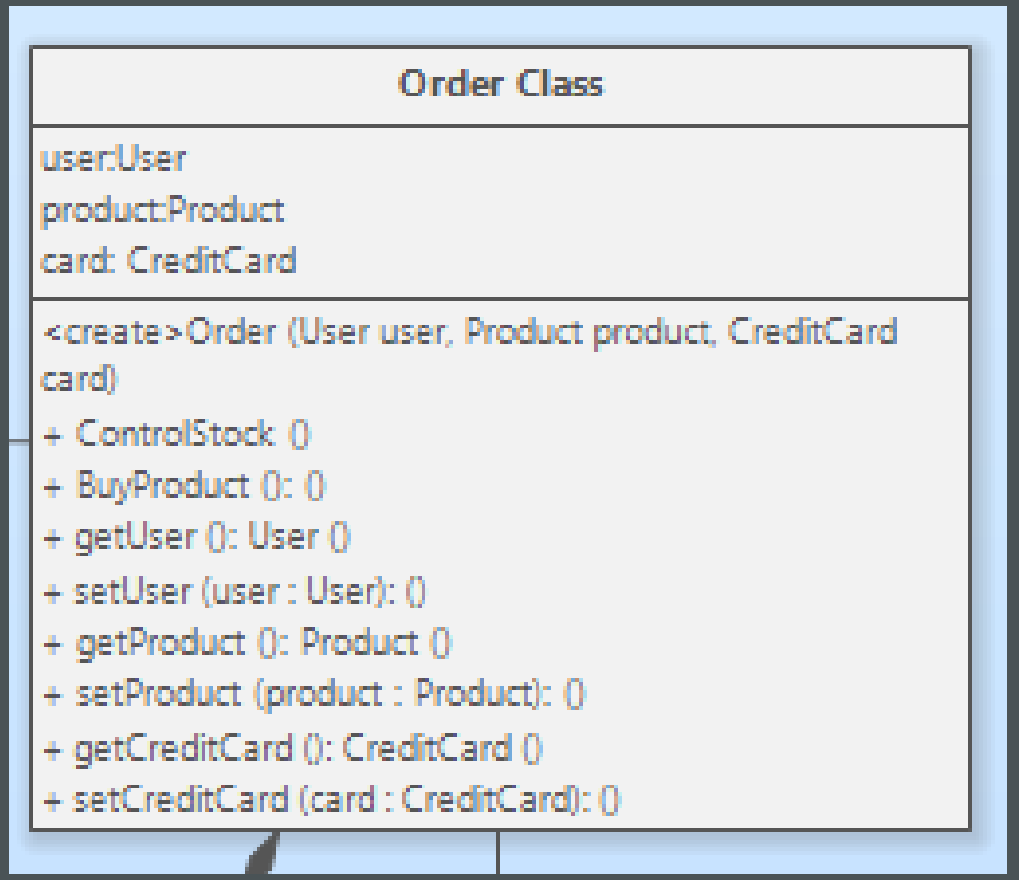
# UML DIAGRAM CREDITCARD CLASS PART

The CreditCard class consists of card holder, card number, card user, security code, and card expiration date. All these components have their own getters and setters in the class. For example, getcardHolder and setcardHolder methods are used to get and set the card holder, while getcardNumber and setcardNumber methods are used to get and set the card number. Additionally, this class has methods like getcardUser and setcardUser to get and set the card user, getsecCode and setsecCode to get and set the security code, and getcardDate and setcardDate to get and set the card expiration date. These methods ensure that all credit card-related data can be accessed and modified as needed, providing a robust framework for managing credit card information in an application.



# UML DIAGRAM ORDER PART

The Order class encompasses a variety of methods to oversee and execute operations. The Controlstock method verifies the current stock status and can issue a warning if stock levels decrease. The BuyProduct method enables the user to make payment for a specified quantity of a product and completes the order. Additionally, the class includes other methods such as getUser and setUser for retrieving and setting user information, getProduct and setProduct for managing product details, and getCreditCard and setCreditCard for handling credit card information.



# CODE PART / USER CLASS

# CODE PART

## FIRST PART OF USER CLASS

The User class is a Java class designed to store user information. This class stores the user's basic details (userName, name, surname, bDay, password, eMail, homeAdress, workAdress) as strings. Additionally, it contains lists of type ArrayList<Product> and ArrayList<CreditCard> to store the user's ordered products, favorite products, and credit card information. The class includes a parameterized constructor that takes parameters such as user name, first name, last name, birth date, password, email address, home address, and work address, assigning these values to the respective variables. It also has a parameterless constructor that creates a user object with default values. This class is designed to maintain user information in an organized manner, facilitating operations related to the user (e.g., placing orders, listing favorite products, storing credit card information).

```java
public class User {
    // Private variables for user information
    private String userName;  // User's username
    private String name;      // User's first name
    private String surname;   // User's last name
    private String bDay;      // User's birth date
    private String password;  // User's password
    private String eMail;     // User's email address
    private String homeAdress;// User's home address
    private String workAdress;// User's work address
    private ArrayList<Product> productOrdered = new ArrayList<Product>()
    private ArrayList<Product> favProducts = new ArrayList<Product>();
    private ArrayList<CreditCard> creditCard = new ArrayList<CreditCard>

    // Constructors with and without parameters
    public User() {
        // Parameterless constructor
    }

    public User(String userName, String name, String surname, String bDa
        this.userName = userName;
        this.name = name;
        this.surname = surname;
        this.bDay = bDay;
        this.password = password;
        this.eMail = eMail;
        this.homeAdress = homeAdress;
        this.workAdress = workAdress;
    }
```

# CODE PART

## SECOND PART OF USER CLASS

The User class contains getter and setter methods to retrieve and update user information. For example, the getUserName() method returns the user's username, while the setUserName(String userName) method updates this information. Other getter and setter methods work similarly. These getter and setter methods ensure the secure management of user information by adhering to the principle of encapsulation within the class.

```java
// Getter and setter methods
public String getuserName() {
    return userName;    // Returns the user's username
}

public void setuserName(String userName) {
    this.userName = userName;    // Sets the user's username
}

public String getname() {
    return name;    // Returns the user's first name
}

public void setname(String name) {
    this.name = name;    // Sets the user's first name
}

public String getsurname() {
    return surname;    // Returns the user's last name
}

public void setsurname(String surname) {
    this.surname = surname;    // Sets the user's last name
}

public String getbDay() {
    return bDay;    // Returns the user's birth date
}

public void setbDay(String bDay) {
    this.bDay = bDay;    // Sets the user's birth date
}

public String getpassword() {
    return password;    // Returns the user's password
}
```

# CODE PART

## THIRD PART OF USER CLASS

Furthermore, methods such as AddCreditCard(CreditCard card), FavoriteProduct(Product product), and Proust(Product product) handle adding a credit card, adding a favorite product, and purchasing a product, respectively. These methods are used to add a new card to the user's credit card list, add a product to the user's favorite products list, and add a product to the list of products ordered by the user. These methods facilitate various user operations and enable effective management of user data.

```
    public ArrayList<Product> getproductOrdered() {
        return productOrdered;     // Returns the list of products ordered by the user
    }

    public void setproductOrdered(ArrayList<Product> productOrdered) {
        this.productOrdered = productOrdered;    // Sets the list of products ordered by the user
    }

    public ArrayList<Product> getfavProducts() {
        return favProducts;     // Returns the user's favorite products list
    }

    public void setfavProducts(ArrayList<Product> favProducts) {
        this.favProducts = favProducts;    // Sets the user's favorite products list
    }

    public ArrayList<CreditCard> getCredtCard() {
        return creditCard;     // Returns the user's credit cards list
    }

    public void setCreditCard(ArrayList<CreditCard> creditcard) {
        this.creditCard = creditcard;    // Sets the user's credit cards list
    }

    // Method to add a credit card
    public void AddCreditCard(CreditCard card) {
        this.creditCard.add(card);    // Adds a new card to the user's credit cards list
    }

    // Method to add a favorite product
    public void FavoriteProduct(Product product) {
        favProducts.add(product);    // Adds a new product to the user's favorite products list
    }

    // Method to purchase a product
    public void PurchaseProduct(Product product) {
        productOrdered.add(product);    // Adds a new product to the list of products ordered by the user
    }
}
```

# CODE PART / CREDIT CARD CLASS

# CODE PART

## FIRST PART OF CREDIT CARD CLASS

The CreditCard class models a credit card with attributes like the cardholder's name, card number, card user, security code, and expiration date. The constructor initializes these fields and validates that the security code is three characters long, printing an error if not. The class includes getter and setter methods for each attribute, allowing for controlled access and modification. This design ensures proper data encapsulation and integrity while maintaining essential functionality.

```
public class CreditCard {
    private String cardHolder;
    private String cardNumber;
    private String cardUser;
    private String secCode;
    private String cardDate;

    // Constructor to initialize CreditCard object
    public CreditCard(String cardHolder, String car
        this.cardHolder = cardHolder;
        this.cardNumber = cardNumber;
        this.cardUser = cardUser;
        this.cardDate = cardDate;

        // Validate security code length
        if (secCode.length() == 3) {
            this.secCode = secCode;
        } else {
            System.out.println("Please enter a vali
        }
    }

    // Getter for cardHolder
    public String getcardHolder() {
        return cardHolder;
    }

    // Setter for cardHolder
    public void setcardHolder(String cardHolder) {
        this.cardHolder = cardHolder;
    }
```

```
// Getter for cardNumber
public String getcardNumber() {
    return cardNumber;
}

// Setter for cardNumber
public void setcardNumber(String cardNumb
    this.cardNumber = cardNumber;
}

// Getter for cardUser
public String getcardUser() {
    return cardUser;
}

// Setter for cardUser
public void setcardUser(String cardUser)
    this.cardUser = cardUser;
}

// Getter for secCode
public String getsecCode() {
    return secCode;
}

// Setter for secCode
public void setsecCode(String secCode) {
    this.secCode = secCode;
}

// Getter for cardDate
public String getcardDate() {
    return cardDate;
}

// Setter for cardDate
public void setcardDate(String cardDate)
    this.cardDate = cardDate;
}
```

# CODE PART / PRODUCT CARD CLASS

# CODE PART

## FIRST PART OF PRODUCT CLASS

The Product class in Java represents a product with attributes (name, color, category, stock, weight, description, price) and methods to get/set attributes and decrease stock.

The Product class in Java defines a product with several attributes, including name, color, category, stock quantity, weight, description, and price. The class includes a constructor to initialize a product object with these attributes, as well as getter and setter methods for each attribute. The class also has a method DecreaseStock to reduce the product stock by a specified amount, if the current stock is sufficient. If the requested amount is greater than the current stock, the method prints a message "Don't have enough product."

```java
public class Product {
    private String pName;
    private String pColor;
    private String pCategory;
    private int pStock;
    private float pWeight;
    private String pDescription;
    private int pPrice;

    // Constructor to initialize Product object
    public Product(String pName, String pColor, String pCategory, int pStock,
    float pWeight, String pDescription, int pPrice) {

        this.pName = pName;
        this.pColor = pColor;
        this.pCategory = pCategory;
        this.pStock = pStock;
        this.pWeight = pWeight;
        this.pDescription = pDescription;
        this.pPrice = pPrice;
    }
// Setter for pStock
public void setpStock(int pStock) {
    this.pStock = pStock;
}

// Getter for pWeight
public float getpWeight() {
    return pWeight;
}

// Setter for pWeight
public void setpWeight(float pWeight) {
    this.pWeight = pWeight;
}
```

```java
// Setter for pDescription
public void setpDescription(String pDescription) {
    this.pDescription = pDescription;
}

// Method to decrease stock by a given amount
public void DecreaseStock(int amount) {
    if (this.pStock >= amount) {
        this.pStock -= amount;
    } else {
        System.out.println("Don't have enough product.");
    }
}
```

# CODE PART / ORDER CARD CLASS

# CODE PART

## FIRST PART OF ORDER CLASS

The Order class in Java represents an order with user, product, and credit card information. It has getters/setters for attributes and methods to manage product stock and process the purchase, displaying an out-of-stock message if necessary.

```java
ublic class order { // Class name should be capitalized
    User user; // User associated with the order
    Product product; // Product being ordered
    CreditCard card; // Credit card used for the purchase

    // Constructor to initialize the order with user, product, and credit card
    order(User user, Product product, CreditCard card) {
        this.user = user;
        this.product = product;
        this.card = card;
    }

    // Getter for the user
    public User getUser() {
        return user;
    }

    // Setter for the user
    public void setUser(User user) {
        this.user = user;
    }
    // Method to control the stock of the product
    public boolean ControlStock() {
        if (product.getpStock() > 0) { // Check if the product is in stock
            product.DecreaseStock(1); // Decrease the stock by 1
            return true; // Return true if the product is available
        } else {
            System.out.println("Product out of stock."); // Print out of stock message
            return false; // Return false if the product is not available
        }
```

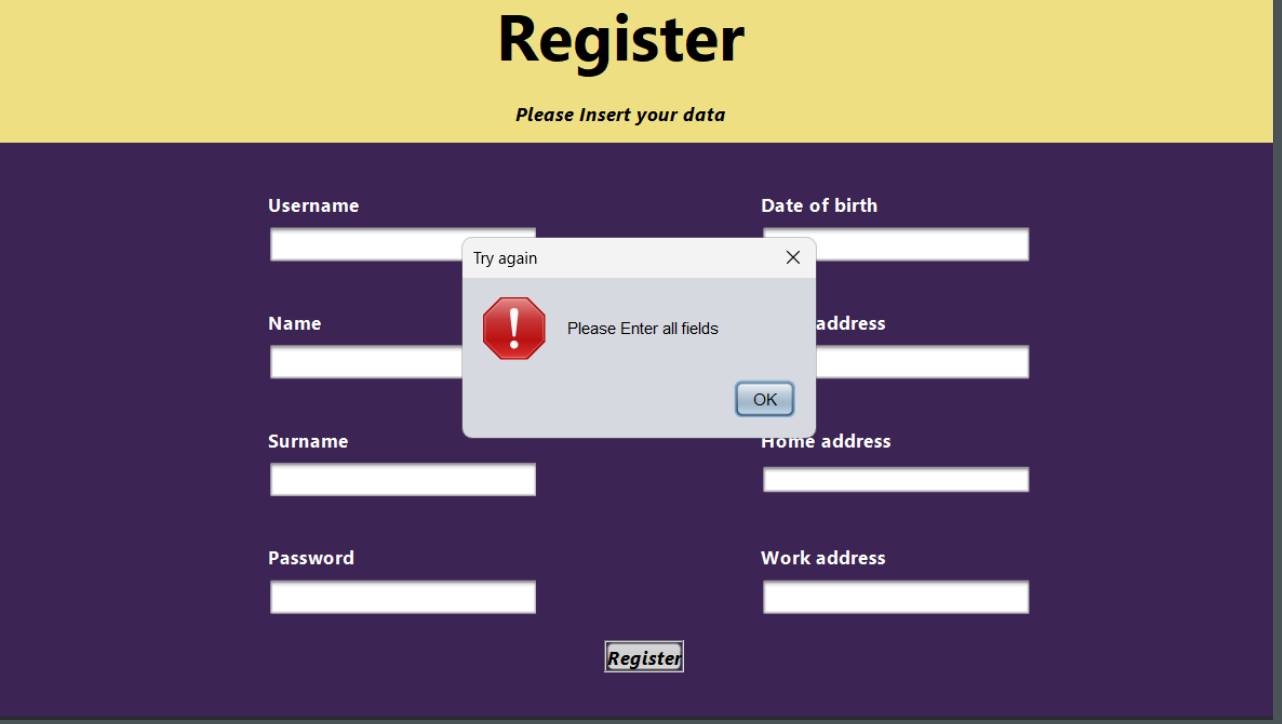
```java
// Method to process the purchase of the product
public void BuyProduct() {
    user.PurchaseProduct(product); // User purchases
}
```

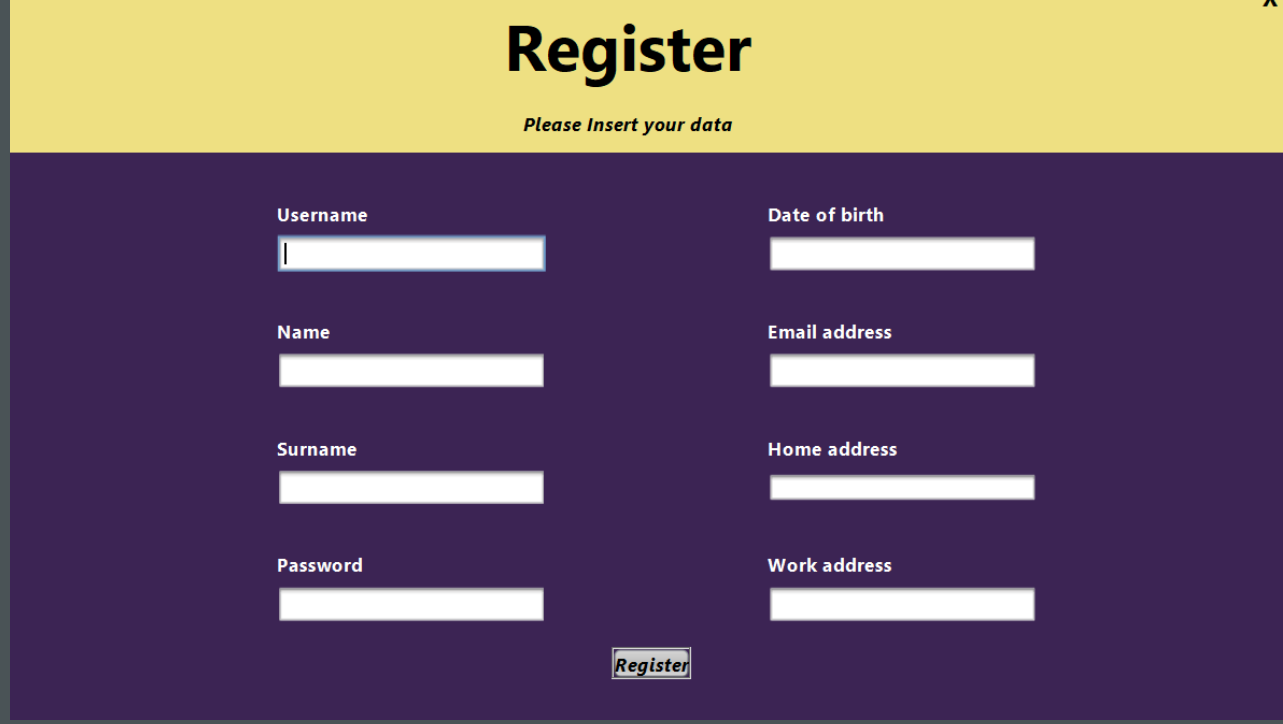
# UI CODE PART / REGISTERUI CLASS

# UI CODE PART
## REGISTER

This code snippet is for a registration process in a Java program. It retrieves user input from various text fields, checks if any fields are empty, and if not, creates a new **User** object with the input data. Then, it transfers the data to a **LoginForm** object, which is displayed to the user. If any fields are empty, it shows an error message using **JOptionPane**. The registration form is hidden once the **LoginForm** is displayed. **It calls the user class(transferData)** and keeps their information while navigating to the Login page.

```
private void tfRegisterActionPerformed(java.awt.event.ActionEvent evt) {

    // Retrieve user input from text fields
    String username = tfUsername.getText();
    String name =  tfName.getText();
    String surname = tfSurname.getText();
    String workadresses = tfWorkaddress.getText();
    String email = tfEmailaddress.getText();
    String homeadresses = tfHomeaddress.getText();
    String password = new String(tfPassword.getPassword());
    String dateofbirth = tfDateofbirth.getText();

    // Check if any field is empty and show an error message if true

    if(username.isEmpty() || name.isEmpty() || surname.isEmpty() || workadresses.isEmpty()
     || email.isEmpty() || homeadresses.isEmpty() || password.isEmpty() || dateofbirth.isEmpty()){
    JOptionPane.showMessageDialog(
    this,
            "Please Enter all fields" ,
            "Try again" ,
            JOptionPane.ERROR_MESSAGE) ;
    } else {
        // Go To Login
        user = new User(username, name, surname, dateofbirth, password, email, homeadresses, workadresses);
        LoginForm lg = new LoginForm();
        lg.transferData(user);
        lg.setVisible(true);
        lg.pack();
        lg.setLocationRelativeTo(null);
        lg.setDefaultCloseOperation(Register.EXIT_ON_CLOSE);
        //Hide the current registration form
        this.setVisible(false);
    }

}
```




# UI CODE PART
## REGISTER VIEW

This page is designed for registration. To complete the registration process, all fields must be filled out. Enter your date of birth in the DD.MM.YYYY format. After filling in all the fields, click the register button to proceed. If any field is left empty, a warning message will appear on the screen. If you have filled in all the fields, you will be taken to the login page.

# UI CODE PART / LOGINUI CLASS

# UI CODE PART
## LOGIN

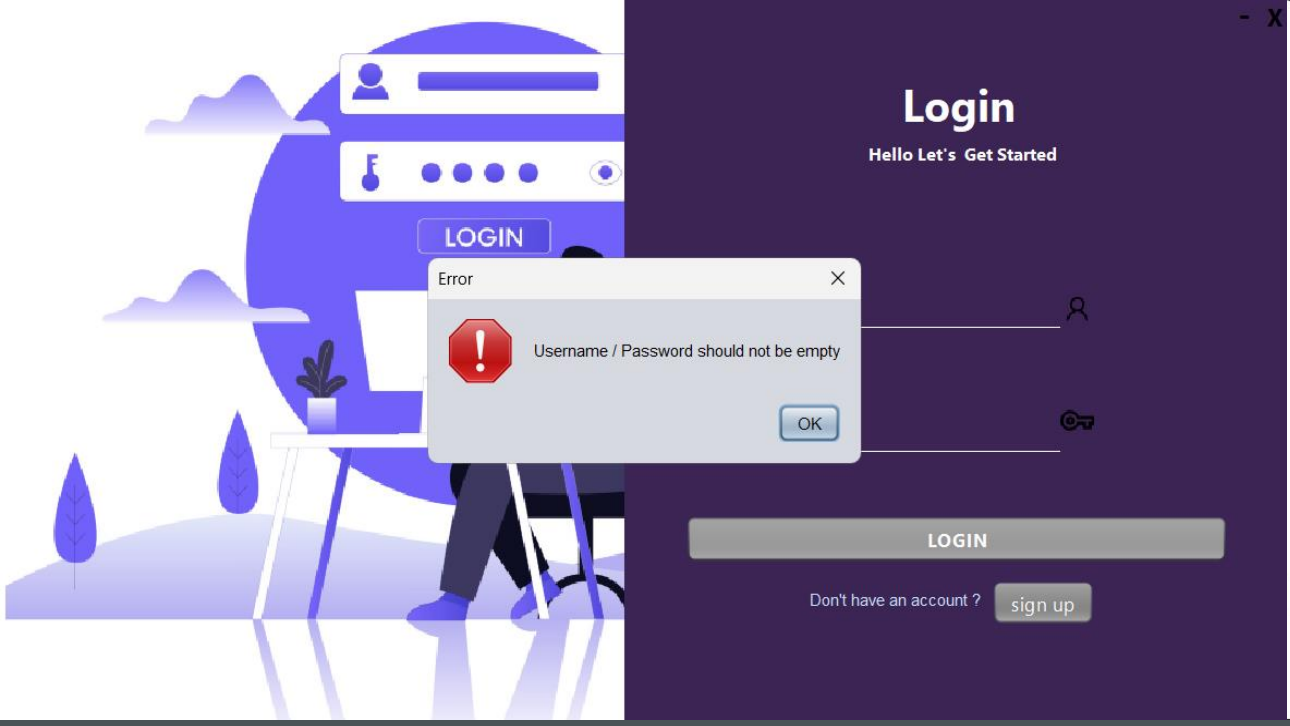
This Java code handles the login button click event in a GUI application. It retrieves the username and password entered by the user, checks if either field is empty, and if not, it validates the credentials against a user object. If the credentials match, it opens the main page of the application and passes the user object to it. If the credentials don't match or the user object is not found, it shows an error message. The code uses JOptionPane to display error messages and JTextField and JPasswordField to retrieve user input.

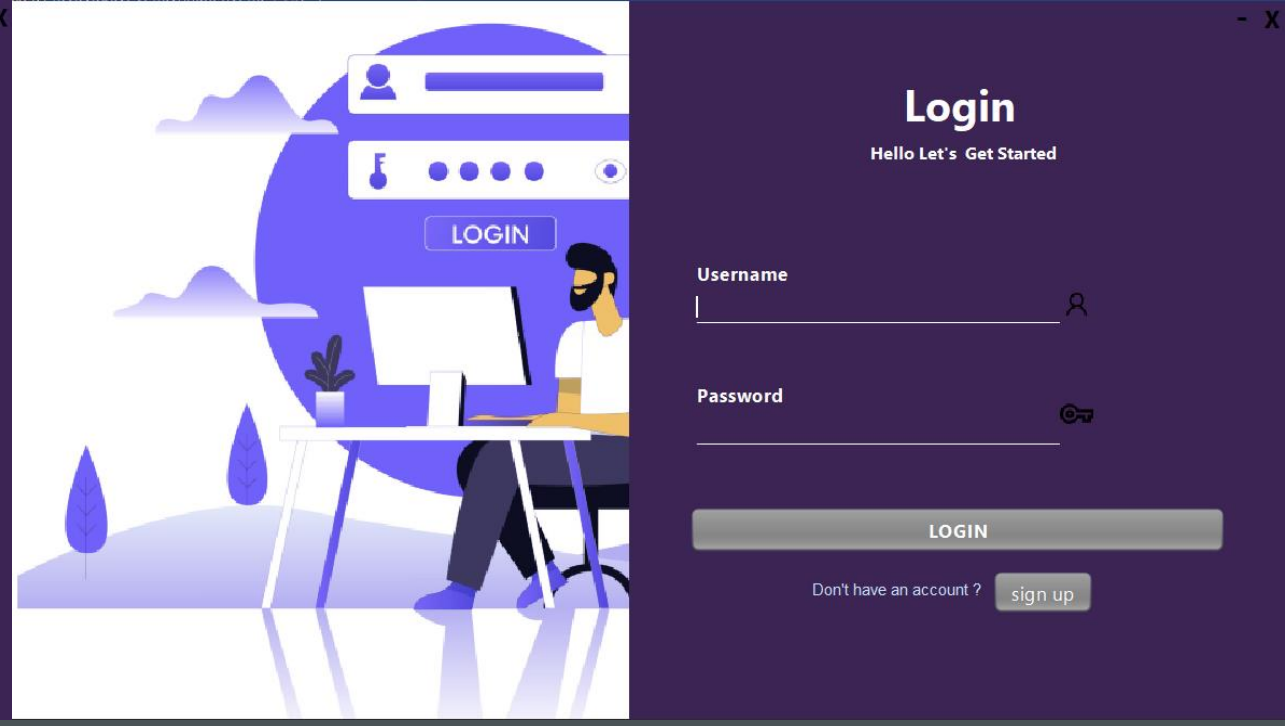
```
  private void bthLoginActionPerformed(java.awt.event.ActionEvent evt) {

/ Get username and password from text fields
      String username = tfUsername.getText();
      String password = new String(tfPassword.getPassword());

/ Check if username or password is empty
   if (username.isEmpty() || password.isEmpty()) {
      // Show error message if either field is empty
      JOptionPane.showMessageDialog(this, "Username / Password should not be empty", "Error", JOptionPane.ERROR_MESSAGE);
   } else {
      // If user is not null
      if (user != null) {
          // Check if the username matches
          if (user.getuserName().equals(username)) {
              if (user.getpassword().equals(password)) {
                  // Successful login actions
                  mainPage mP = new mainPage();
                  mP.GetData(user);
                  mP.setVisible(true);
                  mP.pack();
                  mP.setLocationRelativeTo(null);
                  mP.setDefaultCloseOperation(LoginForm.EXIT_ON_CLOSE);
                  this.dispose(); // Mevcut pencereyi kapat
              } else {
                  // Show error message if the password is wrong
                  JOptionPane.showMessageDialog(this, "Username / Password is Wrong", "Error", JOptionPane.ERROR_MESSAGE);
              }
          } else {
              //  Show error message if the username is wrong
              JOptionPane.showMessageDialog(this, "Username / Password is Wrong", "Error", JOptionPane.ERROR_MESSAGE);
          }
      } else {
          // Show error message if no user is found
          JOptionPane.showMessageDialog(this, "No user found", "Error", JOptionPane.ERROR_MESSAGE);
      }
    }
   }
```

# UI CODE PART
## LOGIN

This Java code is part of a GUI application, specifically a login form. The **formWindowOpened** method creates a fade-in effect when the window is opened. The **jLabel8MouseClicked** method minimizes the window when the "Minimize" label is clicked. The **jButton1ActionPerformed** method opens a new registration form when the "Register" button is clicked. The **main** method starts the login form using
the **EventQueue.invokeLater** method. The **transferData** method is used to transfer user data from another form to this login form. The code handles various events and actions related to the login form, including window opening, minimizing, registering, and transferring user data.

```
private void formWindowOpened(java.awt.event.WindowEvent evt) {
    // Gradually increase window opacity from 0 to 1
    for (double i = 0.0; i <=1.0; i = i+0.1){
    String val = i+ "";
    float f = Float.valueOf(val);
    this.setOpacity(f);
    try{
    Thread.sleep(50); // Wait for 50 milliseconds
    }catch(Exception e){
        // Handle exception
    }
   }
}

private void jLabel8MouseClicked(java.awt.event.MouseEvent evt) {
   // Minimize the window
    this.setState(1);
}

private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
    // Open the registration form
    Register rg = new Register();
       rg.setVisible(true);
       rg.pack();
       rg.setLocationRelativeTo(null);
       rg.setDefaultCloseOperation(Register.EXIT_ON_CLOSE);
       this.setVisible(false);
}

public static void main(String args[]) {
    // Start the login form
   java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            new LoginForm().setVisible(true);
        }
    });
}
// Transfer user data to this form
public void transferData(User u)
{
    this.user = u;
}
```




# UI CODE PART
## LOGIN VIEW

The username and password on this login page must match the username and password entered in the registration form. Otherwise, the application will not proceed to the next page and will display an error message. Additionally, if you leave any fields empty, an error message will also be displayed.

# UI CODE PART / MAINPAGEUI CLASS

# UI CODE PART
## MAINPAGE

The **mainPage** class is a graphical user interface (GUI) component that extends **javax.swing.JFrame**. The class has a constructor that initializes the GUI components and creates three **Product** objects if they haven't been created before. The three products are a Harley-Davidson, a Honda, and a BMW, each with their own attributes such as name, color, type, quantity, price, and description. The products are then added to an **ArrayList** called **ProductList**. The **productsCreated** flag is used to check if the products have already been created, but it is not set to **true** after creation. The **mainPage** class also has a **user** variable, but it is not used in the provided code snippet.

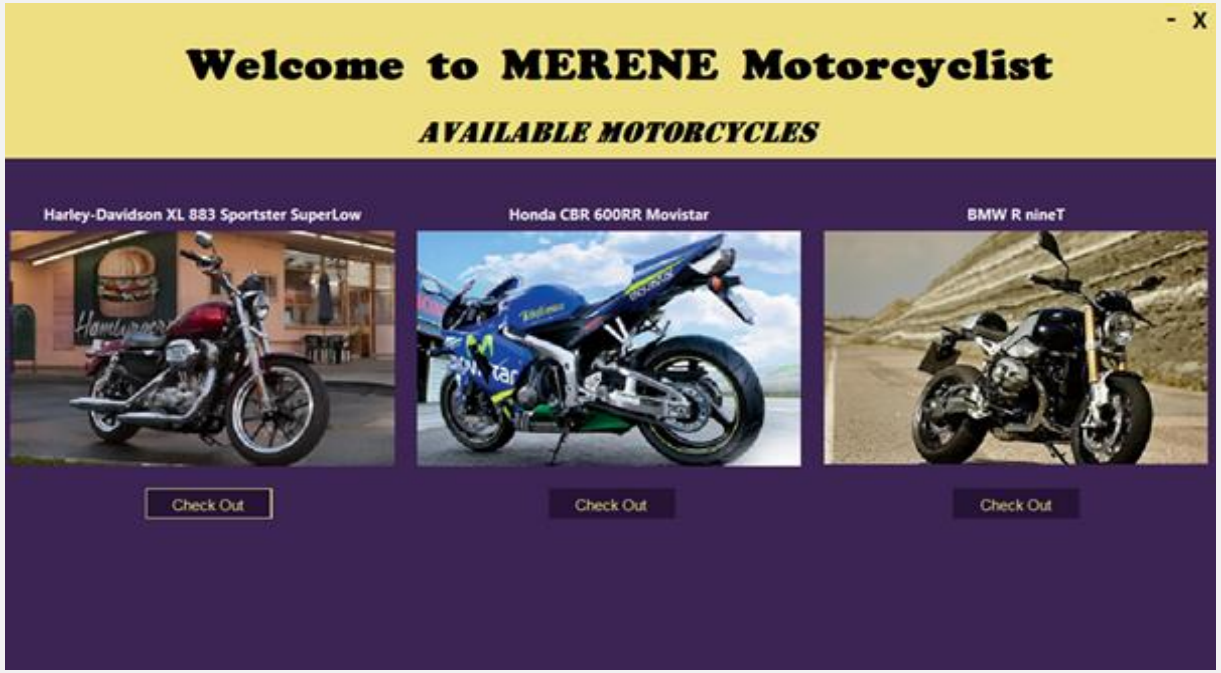
```
*/
public class mainPage extends javax.swing.JFrame {
    User user;
    public boolean productsCreated = false;
    public Product HarleyP;
    public Product HondaP;
    public Product BMWP;
    public ArrayList<Product> ProductList = new ArrayList<Product>();

// Constructor for mainPage
    public mainPage() {
        initComponents();

// Check if products have already been created
        if(productsCreated == false){
            // Initialize Harley-Davidson product
            this.HarleyP = new Product("Harley-Davidson XL 883 Sportster SuperLow", "Red", "Classic", 100, 260,
            "Engine: Air-cooled, Evolution V-Twin engine, 883cc displacement", 20000);

            // Initialize Honda product
            this.HondaP = new Product("Honda CBR 600RR Movistar", "Blue", "Sport", 100, 186,
            "Engine Type: Liquid-cooled, 4-stroke, DOHC, 16-valve inline-four", 15000);

            // Initialize BMW product
            this.BMWP = new Product("BMW R nineT", "Black", "Classic-Sport", 100, 220,
            "Engine: Twin-cylinder, four-stroke, air/oil-cooled boxer engine", 25000);

            // Add products to the ProductList
            ProductList.add(HarleyP);
            ProductList.add(HondaP);
            ProductList.add(BMWP);
        }
    }
```

```
private void HarleyMouseClicked(java.awt.event.MouseEvent evt) {
        // Open HarleyDavidson page and pass user and product data
        HarleyDavidson hD = new HarleyDavidson();
        hD.GetData(user, ProductList);
        hD.setVisible(true);
        hD.pack();
        hD.setLocationRelativeTo(null);
        hD.setDefaultCloseOperation(LoginForm.EXIT_ON_CLOSE);
        this.dispose(); // Close the current window
}

private void HondaMouseClicked(java.awt.event.MouseEvent evt) {
        // Open Honda page and pass user and product data
        Honda h = new Honda();
        h.GetData(user, ProductList);
        h.setVisible(true);
        h.pack();
        h.setLocationRelativeTo(null);
        h.setDefaultCloseOperation(LoginForm.EXIT_ON_CLOSE);
        this.dispose(); // Close the current window
}

private void BMWWMouseClicked(java.awt.event.MouseEvent evt) {
        // Open BMW page and pass user and product data
        BMW b = new BMW();
        b.GetData(user, ProductList);
        b.setVisible(true);
        b.pack();
        b.setLocationRelativeTo(null);
        b.setDefaultCloseOperation(LoginForm.EXIT_ON_CLOSE);
        this.dispose(); // Close the current window
}
```

```
public static void main(String args[]) {
    /* Set the Nimbus look and feel */
    Look and feel setting code (optional)

    /* Create and display the form */
    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            new mainPage().setVisible(true);
        }
    });
}

public void GetData(User user)
{
    // Method to receive user data
    this.user = user;
}

public void GetHarley(Product harley){

// Method to update Harley product data
    ProductList.set(0, harley);
}

// Method to update Honda product data
public void GetHonda(Product hon){
    ProductList.set(1, hon);
}

// Method to update BMW product data
public void GetBMW(Product bmw)
{
    ProductList.set(2, bmw);
}
```

# UI CODE PART
## MAINPAGE

The code defines three mouse click event handlers for Harley, Honda, and BMW buttons, which open new windows for each brand and pass the user and product data to them. The **GetData** method is used to receive user data and store it in the **user** variable. The **GetHarley**, **GetHonda**, and **GetBMW** methods are used to update the product data for each brand in the **ProductList**. The **GetProductList** method is used to update the entire **ProductList**. The **main** method sets the Nimbus look and feel for the GUI and creates an instance of the **mainPage** class, making it visible. When a button is clicked, a new window for the corresponding brand is opened, and the current window is closed. The new window is centered on the screen and has a default close operation set to exit the application when closed.




# UI CODE PART
## MAINPAGE VIEW

- After successfully logging in, you will be directed to the main page where you will see three products, each with limited stock. You can get information about or purchase the product you want by clicking the check-out button. To make a purchase, you must have a credit card.

# UI CODE PART / PRODUCTUI CLASS

# UI CODE PART
## PRODUCTS

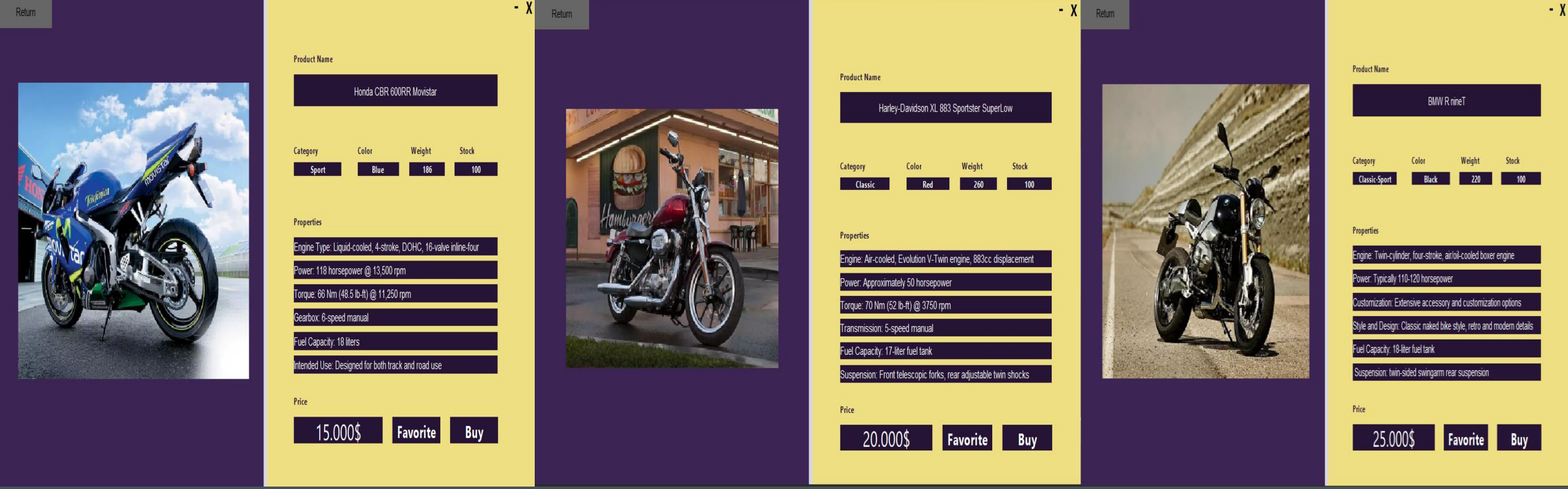
The code creates a new mainPage window, sets its productsCreated variable to true, updates the Harley product data, passes the user data, and displays it in the center of the screen. The current window is closed, and the mainPage window is set to exit the application when closed. It creates a new Checkout window, passes user and product data, and displays it in the middle of the screen when the "buy" button is pressed. The current window is closed and the Payment window is set to exit the application when closed. The payment window likely includes a form for the user to enter payment information. and finally, after making the payment, 1 product will decrease from the product stock.

```
public void GetData(User user, ArrayList<Product> product)
{
    this.user = user;
    this.pList = product;
    jLabel11.setText(pList.get(0).getpStock() + "");
}
```

```
private void jLabel20MouseClicked(java.awt.event.MouseEvent evt) {
                mainPage mP = new mainPage();
                mP.productsCreated = true;
                mP.GetHarley(pList.get(0));
                mP.GetData(user);
                mP.setVisible(true);
                mP.pack();
                mP.setLocationRelativeTo(null);
                mP.setDefaultCloseOperation(LoginForm.EXIT_ON_CLOSE);
                this.dispose(); // Mevcut pencereyi kapat
}
```

```
public class HarleyDavidson extends javax.swing.JFrame {

    /**
     * Creates new form HarleyDavidson
     */
    Product product;
    ArrayList<Product> pList = new ArrayList<Product>();
    User user;
    public HarleyDavidson() {
        initComponents();
    }
```

```
private void buyMouseClicked(java.awt.event.MouseEvent evt) {
                Payment p = new Payment();
                p.GetData(user, pList, 0);
                p.setVisible(true);
                p.pack();
                p.setLocationRelativeTo(null);
                p.setDefaultCloseOperation(LoginForm.EXIT_ON_CLOSE);
                this.dispose(); // Mevcut pencereyi kapat
}
```




# UI CODE PART
## PRODUCTS VIEW

- On the main page, if you click on the check-out section, you will be presented with detailed information about the products. You will be able to see details such as the color, stock quantity, engine type, engine power, and more. If you like the product, you can click the buy button to proceed to the credit card transaction. After entering your credit card information, your order will be accepted.

# UI CODE PART / PAYMENTUI CLASS

# UI CODE PART
## PAYMENT

This Java method handles the "Confirm" button click event in a payment processing system. It retrieves credit card details from text fields, including card number, CVC, holder name, month, and year. The method checks if any of the fields are empty and displays an error message if so. If all fields are filled, it creates a new **CreditCard** object with the user's information. A new **order** object is then created with the user, product, and credit card details. The method checks if the product is in stock using the **ControlStock()** method. If the product is in stock, it completes the purchase and updates the product list. Finally, it displays a success message and opens the main page, passing the user and product list data to it.

```
private void ConfirmMouseClicked(java.awt.event.MouseEvent evt) {
    // Get credit card details from text fields
    String cardnumber = tfcardNumber.getText();
    String cvc = tfCvc.getText();
    String holdername = tfholderName.getText();
    String month = Month.getSelectedItem();
    String year = Year.getSelectedItem();

    // Check if any field is empty
    if(cardnumber.isEmpty() || cvc.isEmpty() || holdername.isEmpty() || month.isEmpty() || year.isEmpty()){
   // Show error message if any field is empty
        JOptionPane.showMessageDialog(this,"Please Enter all fields" ,"Try again" ,JOptionPane.ERROR_MESSAGE) ;

    }else{
        // Create a new CreditCard object
      card = new CreditCard(user.getname() + " " + user.getsurname(), cardnumber, user.getuserName(), cvc, month + " " + year);
      // Create a new order object
      order Order = new order(user, product, card);
      // Check if the product is in stock
      if(Order.ControlStock() == true)
      {
          // Buy the product and update product list
          Order.BuyProduct();
          pList.set(productIndex, product);
           // Show success message
          JOptionPane.showMessageDialog(this,"Order successfully placed" ,"Congratulations" ,JOptionPane.INFORMATION_MESSAGE);
          // Open the main page and pass user and product list data
          mainPage mP = new mainPage();
          mP.GetData(user);
          mP.GetProductList(pList);
          mP.setVisible(true);
          mP.pack();
          mP.setLocationRelativeTo(null);
          mP.setDefaultCloseOperation(LoginForm.EXIT_ON_CLOSE);
          this.dispose(); // Close the current window

      }
      else
      {
          // Show error message if the product is out of stock
          JOptionPane.showMessageDialog(this,"Order out of stock" ,"Sorry" ,JOptionPane.ERROR_MESSAGE) ;
      }
    }
```

# UI CODE PART

## PAYMENT

This Java method handles a mouse click event on a JLabel in a graphical user interface (GUI). It creates a new instance of the mainPage class and sets the productsCreated property to true. The method passes the pList (product list) and user data to the mainPage object using the GetProductList and GetData methods. It then sets the mainPage object to be visible, packs it to fit its contents, and centers it on the screen. The method also sets the default close operation of the mainPage object to EXIT_ON_CLOSE, which means the application will exit when the window is closed. Finally, it closes the current window using this.dispose().

```
private void jLabel20MouseClicked(java.awt.event.MouseEvent evt) {
    // Open the main page
    mainPage mP = new mainPage();
     // Set productsCreated to true to indicate products have already been created
    mP.productsCreated = true;
    // Pass the product list to the main page
    mP.GetProductList(pList);
    // Pass the user data to the main page
    mP.GetData(user);
    // Pass the user data to the main page
    mP.setVisible(true);
    mP.pack();
    mP.setLocationRelativeTo(null);
    mP.setDefaultCloseOperation(LoginForm.EXIT_ON_CLOSE);
    this.dispose(); // Close the current window
}
```

# UI CODE PART

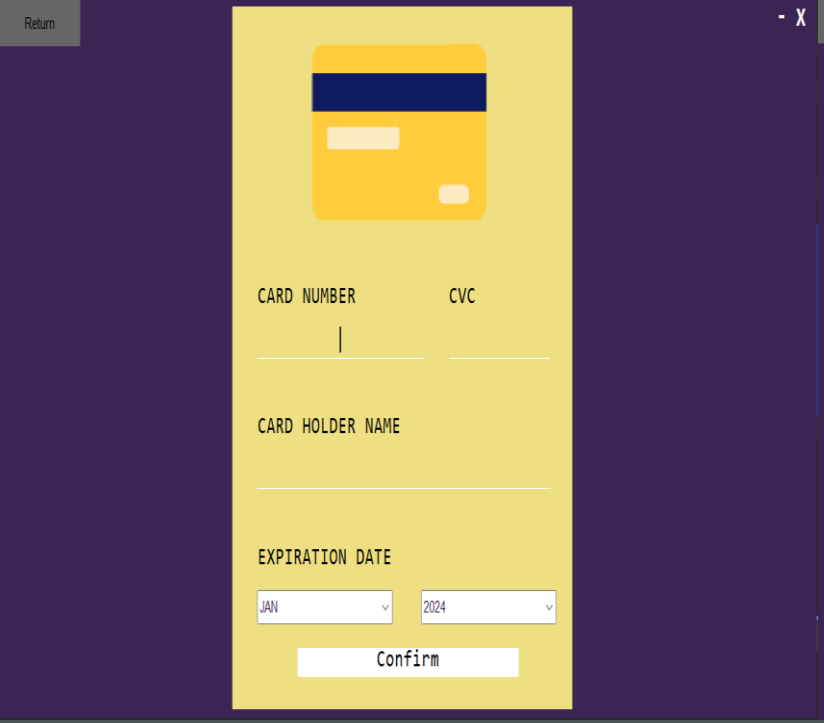
## PAYMENT

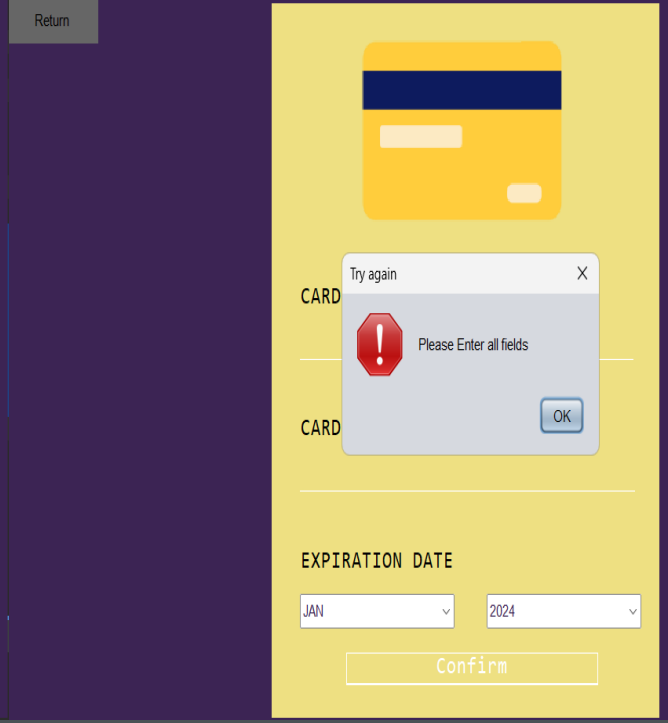
This code is written in Java and consists of two methods: addItem and GetData. The addItem method is used to add items to two combo boxes, Month and Year. The Month combo box is populated with the twelve months of the year from "JAN" to "DEC". The Year combo box is filled with years ranging from "2024" to "2031".

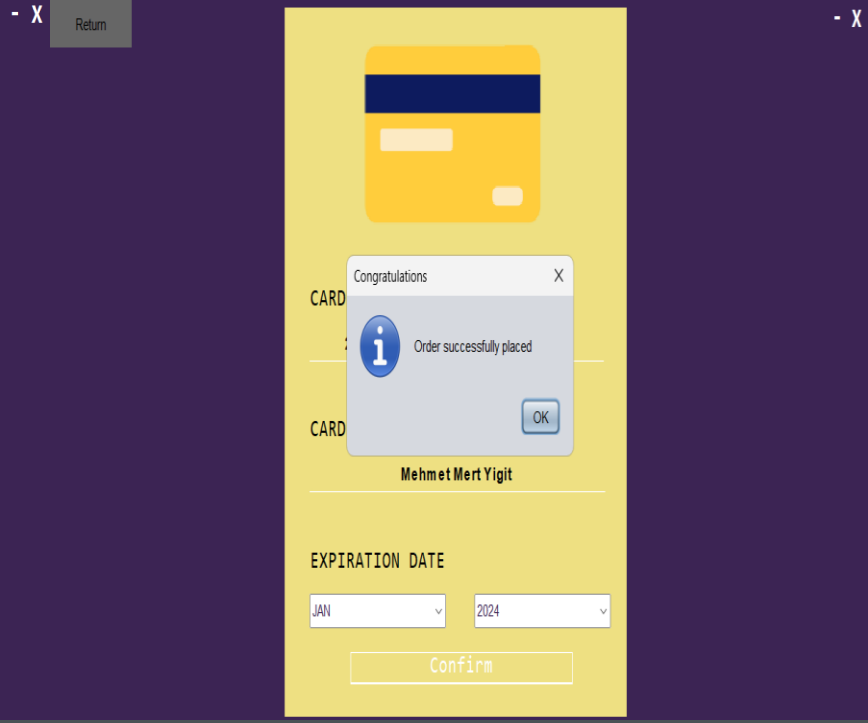
The GetData method is used to set certain data members of the class. It takes three parameters: user, product, and proIndex. The user parameter is an object of the User class, product is an ArrayList of Product objects, and proIndex is an integer. The method sets the class's data members user, pList (product list), productIndex, and product based on the provided parameters.

.

```
public static void main(String args[]) {
    /* Set the Nimbus look and feel */
     Look and feel setting code (optional)
    /* Create and display the form */
    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            new Payment().setVisible(true);
        }
    });
}
public void addItem(){
     // Add months to the Month combo box
    Month.addItem("JAN");
    Month.addItem("FEB");
    Month.addItem("MAR");
    Month.addItem("APR");
    Month.addItem("MAY");
    Month.addItem("JUN");
    Month.addItem("JUL");
    Month.addItem("AUG");
    Month.addItem("SEP");
    Month.addItem("OCT");
    Month.addItem("NOV");
    Month.addItem("DEC");

    // Add years to the Year combo box
    Year.addItem("2024");
    Year.addItem("2025");
    Year.addItem("2026");
    Year.addItem("2027");
    Year.addItem("2028");
    Year.addItem("2029");
    Year.addItem("2030");
    Year.addItem("2031");
}

public void GetData(User user, ArrayList<Product> product, int proIndex)
{
 // Set user data, product list, current product index, and current product
    this.user = user;
    this.pList = product;
    this.productIndex = proIndex;
    this.product = pList.get(productIndex);
}
```




# UI CODE PART
## PAYMENT VIEW

- When we click "Buy" in the product section, it redirects us to the payment page where we need to enter our card details. If we don't enter them, it shows an error; if we do, it confirms the purchase and sends us back to the main menu with a message saying "Purchase successful".